



Meu Primeiro App com o GitHub Copilot ^{v7}

Guia de prompts passo a passo — app de freelancers com avaliação 360°

Node (backend) + React (frontend) · da estrutura ao recrutamento, testes, cobertura e documentação

Projeto Social Garapuvu 2026 · Instrutor: Douglas Adriano Queiroz

Este guia é executado ao vivo: você copia cada prompt no chat do GitHub Copilot (VS Code), **na ordem apresentada**, e mostra o sistema nascendo aos poucos. Ao final você terá uma plataforma de recrutamento completa — CRUD pela interface, máquina de status do projeto, login/logout, avaliação 360°, **testes em todos os níveis da pirâmide**, cobertura de código e testes E2E com dois usuários simultâneos.

✓ **Como o guia está organizado.** Primeiro você prepara o projeto e ensina o contexto ao Copilot (§1–§2). Depois constrói a **fundação** do app na sequência de prompts Fase 0→6 (§6). Em seguida **evolui** para o recrutamento 360° (§7) e o smoke test com dois atores (§8). Por fim, entende os testes (§9), automatiza com o Makefile (§10–§11) e vê como migrar para um banco de dados (§12).

1 Regras de ouro do Copilot

O Copilot é um copiloto: você pilota, ele acelera. A qualidade da resposta depende do seu pedido. Grave estas regras:

- **Dê contexto.** Deixe abertos os arquivos relevantes e use `@workspace` no Copilot Chat.
- **Peça em partes pequenas.** Um endpoint, um componente, um teste por vez.
- **Diga a stack e as regras.** "Node + Express", "status do projeto", "só o dono exclui".
- **Peça os testes junto.** Toda função/rota nova vem com teste.
- **Leia antes de aceitar.** Você assina o código. Não entendeu? `/explain`.
- **Teste e versione a cada passo.** Rode a suíte e faça `git commit` quando algo funcionar.
- **Seletores estáveis para E2E.** Peça `data-testid` nos botões de ação (candidatar, selecionar, excluir, enviar-avaliação, **sair**).
- **Cubra os dois lados de cada regra.** Fez login? Teste o logout. Salvou no `localStorage`? Teste que ele é limpo.

📌 **Dica de comandos do Copilot Chat:** `/explain` explica um trecho, `/fix` sugere correção, `/tests` gera testes para a seleção, e `@workspace` considera todo o projeto no contexto.

2 Contexto e regras para o Copilot (arquivo `.md`)

Antes de pedir código, ensine o Copilot **de uma vez** sobre o projeto. O jeito mais poderoso é criar um arquivo que o Copilot **lê automaticamente** em todo chat deste repositório:

```
# na raiz do projeto (pasta app/)
mkdir -p .github
# crie o arquivo .github/copilot-instructions.md com o conteúdo abaixo
```

Salve este conteúdo em `.github/copilot-instructions.md` — ele vira o contexto permanente do Copilot:

```
# Instruções do projeto para o GitHub Copilot

## O que é o projeto
FreelaAvalia 360 — plataforma de freelancers com avaliação 360° entre freelancer e
contratante. Contratantes publicam projetos abertos; freelancers se candidatam; o
contratante seleciona; ao final, os dois se avaliam (nota 1-5 + comentário).

## Stack
- Backend: Node + Express, dados em memória (arrays), sem banco.
- Frontend: React + Vite, mobile-first, sessão no localStorage.
- Testes: Vitest, Supertest, React Testing Library, Playwright; cobertura com provider v8.
- Estrutura: pasta app/ com backend/ e frontend/ lado a lado.

## Regras de negócio (não quebrar)
- Usuário: e-mail único e válido; papel freelancer/contratante; senha >= 4; a senha NUNCA sai nas
respostas.
- Projeto: nasce "aberto"; ordem aberto -> em_aprovacao -> em_andamento -> concluido; só o dono
edita/exclui e só enquanto aberto; concluído nunca é excluído.
- Candidatura: 1 por freelancer por projeto; só em projeto aberto.
- Avaliação: só em em_andamento/concluido; nota inteira 1-5; 1 por lado; entre as partes do contrato.

## Como responder aqui
- Peça em partes pequenas; gere o teste junto (feliz + bordas).
- Valide entradas e devolva status HTTP corretos (200/201/400/401/404).
- Separe app.js (exporta a app) de server.js (listen). Comente com JSDoc.
- data-testid nos botões de ação. Nada de senhas/segregos no código (.env + .gitignore).
- Mocks só na borda (rede/relógio/navegador), nunca a regra de negócio testada.
```

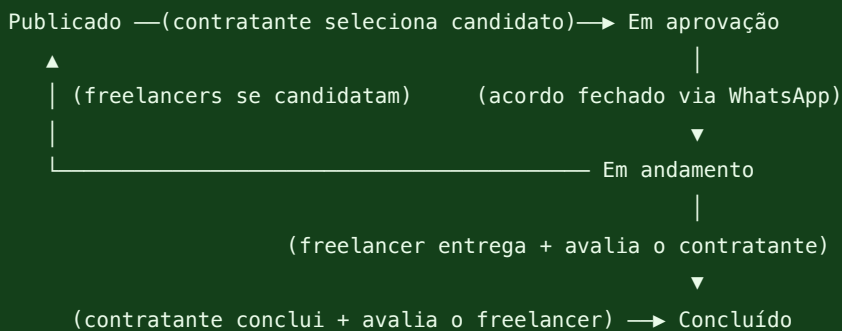
✓ **Por que isso ajuda.** Com o `copilot-instructions.md` no lugar, você não precisa repetir a stack e as regras a cada prompt — o Copilot já "sabe" o projeto. Os prompts das próximas seções ficam curtos porque o contexto já está dado.

3 O app: FreelaAvalia 360 (recrutamento 360°)

Contratantes publicam projetos abertos; freelancers se candidatam; o contratante vê os candidatos com a reputação de cada um e seleciona; ao final, os dois se avaliam. A avaliação mútua (1 a 5 + comentário) é a **avaliação 360°**.

Entidade	O que guarda	Regras principais
Usuário	nome, e-mail, papel, senha, telefone e endereço (opcionais)	e-mail único; papel válido; senha ≥ 4; senha nunca sai nas respostas
Projeto	título, descrição, contratante, freelancer (opcional), contato, status	nasce aberto; percorre a máquina de status; só o dono edita/exclui
Candidatura	projeto + freelancer inscrito	1 por freelancer por projeto; só em projeto aberto
Avaliação	de quem, para quem, nota (1–5), comentário	durante o andamento ou após concluir; 1 por lado

A máquina de status do projeto



 **Regras do MVP:** projeto em andamento não pode ser editado nem reaberto — para mudar, publique um novo. Editar/Excluir só aparecem enquanto o projeto está **Publicado**. A negociação de valores acontece no WhatsApp, fora do app; o contratante apenas registra quando fecha o acordo.

4 Pré-requisitos e bibliotecas

Antes dos prompts, confirme que tudo está instalado. Abra o terminal do VS Code e rode:

```

$ node -v      # deve mostrar v18, v20 ou superior
$ npm -v      # o gerenciador de pacotes do Node
$ git --version
$ code -v     # confirma o VS Code na linha de comando
  
```

✓ **Como saber se passou:** cada comando deve imprimir um número de versão (ex.: `v20.11.0`). Se aparecer "command not found", instale a ferramenta antes de seguir. Confirme também a extensão **GitHub Copilot** instalada e logada.

Tecnologia	Para que serve	Instalação
Express	API e endpoints do backend	<code>npm i express</code>
Vite + React	Base do frontend (telas)	<code>npm create vite@latest</code>
Vitest	Testes unitários e de integração	<code>npm i -D vitest</code>
Supertest	Testa a API (requisições nos endpoints)	<code>npm i -D supertest</code>
React Testing Library	Testa componentes de tela	<code>npm i -D @testing-library/react @testing-library/jest-dom</code>
Playwright	Testes E2E (fluxo real no navegador)	<code>npm i -D @playwright/test</code>
@vitest/coverage-v8	Mede a cobertura de testes	<code>npm i -D @vitest/coverage-v8</code>

5 A pirâmide de testes

 **Analogia da obra:** os testes são a **fiscalização de qualidade** em camadas. **Unitário** = testar uma peça isolada (rápido, base da pirâmide). **Integração** = uma parede montada (peças juntas). **Componente/E2E** = percorrer a casa pronta como um morador. **Cobertura** = quanto da obra o fiscal realmente inspecionou.

Nível	No FreelaAvalia 360	Ferramenta
Unitário (base)	validar nota, média, e-mail, <code>podeAvaliar</code> ; repositório	Vitest
Integração (meio)	API funcionando: cria usuário? salva avaliação? devolve a média?	Vitest + Supertest
Componente	telas isoladas (Estrelas, Auth, Projetos...)	React Testing Library
E2E (topo)	fluxo real no navegador, com dois atores	Playwright

Escrevemos **muitos** testes de unidade (rápidos e baratos), **alguns** de integração e **poucos** de ponta a ponta. Assim a suíte é rápida e confiável.

6 A sequência de prompts — construindo o app (execute nesta ordem)

A ordem importa: o Copilot rende muito melhor quando o projeto já tem estrutura e ele "vê" os arquivos anteriores. Faça um `git commit` ao fim de cada fase.

Fase 0 — Preparar o projeto

```
$ mkdir freelavalia && cd freelavalia
$ git init
$ npm init -y
$ code . # abre o VS Code nesta pasta
```

PROMPT 0.1 — ESTRUTURA E README

@workspace Vamos criar um app chamado FreelaAvalia 360: uma plataforma de freelancers com avaliação 360° entre freelancer e contratante. Backend em Node + Express, frontend em React + Vite, testes com Vitest, Supertest, React Testing Library e Playwright. Proponha uma estrutura de pastas simples (backend/ e frontend/) e um README inicial explicando o projeto. Ainda não escreva código de funcionalidade – só a estrutura e o README.

✓ **Verificar:** foram criadas as pastas backend/ e frontend/ e um README.md . Leia o README: ele descreve o app corretamente?

Fase 1 — Backend: modelo e regras (lógica pura = testes unitários)

```
$ cd backend && npm init -y
$ npm i express
$ npm i -D vitest supertest @vitest/coverage-v8
```

PROMPT 1.1 — REGRAS DE NEGÓCIO ISOLADAS

@workspace Crie o arquivo backend/src/regras.js com funções puras (sem banco, sem Express) e comentários JSDoc explicando cada uma:

- validarNota(nota): retorna true se for inteiro entre 1 e 5.
- emailValido(email): valida formato básico de e-mail.
- mediaAvaliacoes(lista): recebe uma lista de notas e retorna a média com 1 casa decimal (0 se vazia).
- podeAvaliar(contrato): permite avaliar se o status for "em_andamento" ou "concluido".

Explique cada função em uma frase.

PROMPT 1.2 — TESTES UNITÁRIOS DESSAS REGRAS

@workspace Gere testes unitários com Vitest para backend/src/regras.js, no arquivo backend/src/regras.test.js. Prefixe o describe com "@unitario". Cubra casos válidos, inválidos e de borda: nota 0, 1, 5, 6 e valores não inteiros; e-mail com e sem @; média de lista vazia e de várias notas; podeAvaliar para "aberto", "em_andamento" e "concluido".

```
$ npx vitest run # roda os testes uma vez
$ npx vitest run --coverage # roda + relatório de cobertura
```

✓ **Verificar:** o terminal mostra os testes passando (verde) e regras.js perto de 100%. **Boa prática:** testar as bordas (nota 0 e 6) é o que pega a maioria dos bugs.

Fase 2 — Backend: a API (endpoints = testes de integração)

PROMPT 2.1 — ARMAZENAMENTO SIMPLES E SEPARADO

@workspace Crie backend/src/repositorio.js: um armazenamento em memória (objetos/arrays) para usuarios, contratos e avaliacoes, com funções de criar e buscar e um reset() para os testes. Mantenha-o separado da API (para facilitar os testes). Comente cada função com JSDoc.

PROMPT 2.2 — A APLICAÇÃO EXPRESS

```
@workspace Crie backend/src/app.js exportando uma app Express (sem dar app.listen aqui –
isso facilita os testes). Use as regras de regras.js e o repositorio.js. Habilite CORS.
Endpoints:
- POST /usuarios (cria usuário; e-mail único e válido; papel "freelancer" ou "contratante"; senha >=
4; nunca retorne a senha)
- POST /login (autentica por e-mail e senha; retorna o usuário sem a senha)
- POST /contratos (cria um projeto entre contratante e freelancer)
- PATCH /contratos/:id/concluir (muda status para "concluido")
- POST /avaliacoes (só se em_andamento/concluido; nota 1–5; 1 por lado)
- GET /usuarios/:id (retorna o usuário com a média de avaliações recebidas)
Valide as entradas e retorne os status HTTP corretos (201, 400, 401, 404). Comente cada rota.
```

PROMPT 2.3 — SEPARAR O SERVIDOR

```
@workspace Crie backend/src/server.js que importa a app de app.js e chama
app.listen(3001). Adicione no package.json os scripts "start": "node src/server.js",
"test": "vitest run" e "coverage": "vitest run --coverage".
```

PROMPT 2.4 — TESTES DE INTEGRAÇÃO DA API

```
@workspace Gere testes de integração com Vitest + Supertest em backend/src/app.test.js,
importando a app de app.js. Prefixe o describe com "@integracao". Teste o fluxo feliz e os
erros: criar usuários, login certo e errado, criar contrato, tentar avaliar antes da hora
(400), concluir, avaliar dos dois lados, e conferir que GET /usuarios/:id retorna a média
correta. Inclua um teste de e-mail duplicado (400).
```

```
$ npx vitest run --coverage
```

✓ **Verificar:** todos os testes passam e a cobertura de `app.js` sobe. Teste também "à mão": rode `npm start` e, no Postman/Insomnia, faça um `POST /usuarios`. **Boa prática:** separar app de server deixa a API testável sem subir a porta.

Fase 3 — Frontend: React + Vite (mobile-first)

📱 **Mobile-first** significa projetar primeiro para a tela pequena (celular) e depois crescer para telas maiores. Como arrumar uma mala pequena: leva-se só o essencial e bem organizado; numa mala maior é fácil espalhar com folga. O CSS mobile-first escreve o estilo base do celular e usa `@media (min-width: ...)` para adicionar ajustes em telas maiores.

```
$ cd ../frontend
$ npm create vite@latest . -- --template react
$ npm i
$ npm i -D vitest @testing-library/react @testing-library/jest-dom jsdom @vitest/coverage-v8
```

PROMPT 3.0 — BASE DE ESTILO MOBILE-FIRST

@workspace Configure o frontend com abordagem MOBILE-FIRST. Em frontend/src/index.css: defina os estilos base pensando primeiro no celular (~360px) – fonte legível, botões com mín. 44px de altura, espaçamentos confortáveis e conteúdo em uma coluna. Depois use @media (min-width: 768px) e (min-width: 1024px) apenas para ADICIONAR colunas em telas maiores. Paleta Garapuvu: verde #2E7D32, verde-escuro #14401A, amarelo #F9A825, fundo claro #F1F8E9, texto #1B3A1B. Comente o que cada breakpoint faz.

PROMPT 3.1 — COMPONENTE DA NOTA (ESTRELAS)

@workspace Crie frontend/src/components/Estrelas.jsx: um componente React que recebe a prop "valor" (1 a 5) e mostra estrelas preenchidas/vazias, e uma prop opcional onChange para selecionar a nota clicando (e somente leitura para desabilitar). Comente as props. Não use bibliotecas externas.

PROMPT 3.2 — TELA DE AVALIAÇÃO CONSUMINDO A API

@workspace Crie frontend/src/pages/Avaliar.jsx: um formulário para enviar uma avaliação 360 (escolher nota com o componente Estrelas + comentário) que faz POST em http://localhost:3001/avaliacoes. Trate carregando, sucesso e erro (mensagens claras). Use fetch. Comente o fluxo.

PROMPT 3.3 — PERFIL COM A MÉDIA RECEBIDA

@workspace Crie frontend/src/pages/Perfil.jsx que busca GET /usuarios/:id e mostra o nome, o papel e a média de avaliações usando o componente Estrelas. Trate carregando e erro.

✓ **Verificar:** rode npm run dev (frontend) e npm start (backend, noutro terminal). A tela de avaliar envia e mostra sucesso? O perfil mostra a média? **Boa prática:** sempre tratar carregando e erro, não só o "caminho feliz".

Fase 4 — Testes de interface e E2E

PROMPT 4.1 — TESTE DE COMPONENTE (INTERFACE)

@workspace Gere um teste com Vitest + React Testing Library em frontend/src/components/Estrelas.test.jsx (descreva com "@interface"): renderiza com valor 3 e verifica 3 estrelas preenchidas; simula clique na 5ª estrela e verifica onChange(5). Configure o Vitest para usar o ambiente jsdom.

PROMPT 4.2 — TESTE E2E (FLUXO COMPLETO)

@workspace Crie um teste E2E com Playwright em frontend/e2e/avaliacao.spec.js que abre a tela de avaliar, seleciona a nota 5, escreve um comentário, envia e verifica a mensagem de sucesso. Crie um playwright.config.js apontando baseURL para http://localhost:5173.

```
$ npm run vitest run --coverage # componentes + cobertura do front
$ npm i -D @playwright/test && npm run playwright install chromium
```

✓ **Verificar:** `npx vitest run` e `npx playwright test` passam. **Boa prática:** use seletores estáveis (`data-testid`) para o robô não quebrar quando o visual mudar.

Fase 5 — Cobertura e CI

PROMPT 5.1 — RELATÓRIO DE COBERTURA LEGÍVEL

@workspace Configure o Vitest (`vite.config.js`) para gerar cobertura com o provider "v8" nos formatos "text" e "html" (pasta `coverage/`). Defina thresholds mínimos (ex.: 60% de linhas e funções). Explique, em comentário, o que cada limite significa.

PROMPT 5.2 — RODAR TUDO NO GITHUB ACTIONS

@workspace Crie `.github/workflows/ci.yml` que, a cada push, instala as dependências e roda os testes com cobertura do backend e do frontend. Se a cobertura ficar abaixo do limite, o build deve falhar.

🕒 **Cuidado com a métrica.** Cobertura mostra o que faltou testar — não prova ausência de bugs. 100% verde com asserção fraca não vale nada.

Fase 6 — Documentação

PROMPT 6.1 — DOCUMENTAR MÉTODOS E API

@workspace Revise `regras.js`, `repositorio.js` e `app.js` e garanta que TODA função e TODA rota têm JSDoc (o que faz, parâmetros, retorno). Atualize o README com: o que é o app, tecnologias, como instalar, como rodar (backend e frontend), como executar os testes e ver a cobertura, e uma tabela dos endpoints da API.

✓ **Verificar:** o README permite que outra pessoa rode o projeto do zero seguindo só ele. **Boa prática:** documentação errada engana — revise se ela bate com o código.

7 Evoluindo para o recrutamento 360° (CRUD + candidaturas + status)

Com a fundação pronta, transformamos o projeto em uma plataforma de recrutamento completa. Faça `git commit` após cada prompt.

PROMPT 7.1 — PROJETO ABERTO + CONTATO NO PERFIL

@workspace Ajuste o modelo de PROJETO para ser um anúncio ABERTO: o freelancer passa a ser opcional na criação (nasce `null` = "Aguardando freelancer"). Adicione ao usuário os campos OPCIONAIS `endereco` e `telefone`. No POST `/contratos`, guarde `descricao` e o contato (`email/endereco/telefone`) do contratante — puxando do perfil quando não vierem no corpo. Atualize os testes de integração.

PROMPT 7.2 — EDITAR E EXCLUIR (PATCH E DELETE)

@workspace No backend, crie PATCH /contratos/:id (edita titulo/descricao/contato; título não pode ficar vazio) e DELETE /contratos/:id. No frontend, transforme o modal "Novo projeto" num formulário reutilizável (criar E editar) com descrição (textarea) e telefone/endereço pré-preenchidos do perfil. Nos cards, mostre descrição e contato e adicione Editar e Excluir (com confirmação) só para o dono.

PROMPT 7.3 — EDITAR O PRÓPRIO PERFIL

@workspace Crie PATCH /usuarios/:id (edita nome/telefone/endereco; nunca expõe a senha) e, na tela Perfil.jsx, além da reputação, adicione um formulário para editar nome, telefone e endereço. Ao salvar, atualize o usuário logado (e o localStorage). Associe cada label ao input com htmlFor/id (acessibilidade e testabilidade).

PROMPT 7.4 — CANDIDATURAS (FREELANCER SE INSCREVE)

@workspace Crie a entidade CANDIDATURA no repositório (projeto + freelancer) e os endpoints: POST /contratos/:id/candidaturas (só freelancer, projeto aberto, sem duplicar) e GET /contratos/:id/candidaturas (lista os candidatos com nome e a MÉDIA das avaliações). Inclua os ids dos candidatos no GET /contratos para o front montar os botões.

PROMPT 7.5 — MÁQUINA DE STATUS (SELEÇÃO → ANDAMENTO → CONCLUSÃO)

@workspace Implemente as transições, validando a ordem:

- PATCH /contratos/:id/selecionar {freelancerId}: só entre os candidatos → "em_aprovacao";
- PATCH /contratos/:id/andamento: confirma o acordo → "em_andamento";
- PATCH /contratos/:id/concluir: só a partir de "em_andamento" → "concluido".

Ajuste podeAvaliar para liberar em "em_andamento" e "concluido". Cubra tudo com testes de integração (inclusive as ordens inválidas devolvendo 400).

PROMPT 7.6 — TELAS POR PAPEL

@workspace Em Projetos.jsx, mostre ações conforme o papel e o status:

- Freelancer: vê os projetos abertos e "Candidatar-se"; quando selecionado e em andamento, "Finalizar trabalho e enviar feedback" (avaliação 360 para o contratante).
- Contratante: "Ver candidatos (N)" abre um modal com a reputação (estrelas) e "Selecionar"; depois "Fechei o acordo → iniciar" e, quando o freelancer entregar, "Concluir e avaliar". Use data-testid nos botões de ação.

PROMPT 7.7 — REGRAS DE EXCLUSÃO DE PROJETO

@workspace Refine o DELETE /contratos/:id: projeto CONCLUÍDO nunca pode ser excluído; só projetos PUBLICADOS (aberto) podem ser excluídos; se houver candidatos inscritos, remova-os antes. Crie DELETE /contratos/:id/candidaturas/:freelancerId (pelo próprio freelancer OU pelo contratante). No front, adicione "Retirar candidatura" e "Remover" no modal. Teste as três regras.

PROMPT 7.8 — LOGOUT (E2E + UNIDADE)

@workspace Crie frontend/e2e/logout.spec.js: cadastra/entra, confirma o app e a chave freelavalia_user no localStorage, clica em data-testid="btn-sair", verifica que voltou para a Landing, que a chave foi removida e que um reload NÃO reloga sozinho. Some um teste de unidade em src/App.test.jsx mockando as páginas para validar login (grava no localStorage), logout (limpa) e a restauração da sessão ao montar.

✓ **Verificar:** abra duas janelas (uma logada como contratante, outra como freelancer). Publique → candidate-se → selecione → inicie → freelancer avalia → contratante conclui e avalia. Tente excluir um projeto concluído: deve ser bloqueado com mensagem clara.

8 Smoke test E2E com dois atores

🔗 **Analogia da obra:** o smoke test com dois atores é o **ensaio geral com moradores de verdade**: um contratante e um freelancer usam o prédio ao mesmo tempo, em janelas separadas, e a gente confere o ciclo completo — da publicação da vaga à avaliação final.

PROMPT 8.1 — CONFIG COM SLOWMO POR VARIÁVEL DE AMBIENTE

@workspace Em frontend/playwright.config.js, aponte baseUrl para http://localhost:5173 e configure use.launchOptions.slowMo lendo process.env.SLOWMO (0 por padrão). Assim o mesmo teste roda rápido no CI (headless) e devagar na aula (headed).

PROMPT 8.2 — SMOKE TEST DO FLUXO COMPLETO (2 ATORES)

@workspace Crie frontend/e2e/fluxo-completo.spec.js. Use { browser } e crie DOIS contextos (contratante e freelancer). Passos: contratante cadastra e publica a vaga → freelancer cadastra, vê a vaga e se candidata → contratante abre "Ver candidatos", seleciona → "Fechei o acordo → iniciar" → freelancer "Finalizar trabalho", 5 estrelas e envia → contratante "Concluir e avaliar", 5 estrelas e envia → verifica o status "Concluído". Use getByTestId e recarregue a página ao trocar de ator.

PROMPT 8.3 — SCRIPT NPM DEDICADO E WEBSERVER AUTOMÁTICO

@workspace No package.json do frontend, adicione "e2e:smoke": "SLOWMO=900 playwright test fluxo-completo --headed --workers=1". Em playwright.config.js, adicione a chave webServer com DOIS serviços: backend (command "npm start", cwd "../backend", url http://localhost:3001/contratos) e frontend (command "npm run dev", url http://localhost:5173), ambos com reuseExistingServer: !process.env.CI e timeout 30000.

```
cd frontend
npm run e2e          # todos os E2E (headless) – sobe os serviços sozinho
npm run e2e:smoke    # só o fluxo completo, headed + slowMo
```

9 Catálogo de cenários por tipo de teste

 **Analogia da obra:** aqui abrimos a pasta e lemos *o que* cada laudo verifica. Em cada nível testamos três famílias: o **caminho feliz** (funciona), as **bordas e exceções** (o que dá errado — e a mensagem certa aparece?) e a **resiliência/segurança** (o sistema se protege sozinho?).

9.1 Unitário — a peça isolada (`regras.js` , `repositorio.js` , `semSenha`)

Cenário	Família	Por que importa
<code>validarNota</code> aceita 1 a 5 e rejeita 0, 6, negativos e decimais	borda	A nota é o coração da avaliação 360.
<code>emailValido</code> aceita bem formado e recusa sem <code>@</code> /sem domínio	borda	Barra cadastro inválido antes do banco.
<code>mediaAvaliacoes</code> retorna 0 para lista vazia e média com 1 casa	borda + feliz	Evita divisão por zero.
<code>podeAvaliar</code> libera em andamento/concluído e nega antes	feliz + borda	Garante a ordem da máquina de status.
<code>semSenha</code> remove a senha e não quebra com <code>null</code> / <code>undefined</code>	segurança	A senha nunca pode vazar.

9.2 Integração — a parede montada (`app.test.js` via Supertest, sem mocks)

Cenário	Família	Por que importa
Cria usuário válido (201) e a resposta não traz a senha	feliz + segurança	Contrato público seguro.
Rejeita e-mail duplicado, papel inválido e senha curta (400)	borda	As três regras de cadastro.
Login recusa e-mail inexistente (401) sem revelar se existe	segurança	Não dar pista a quem adivinha contas.
Não exclui projeto com candidatos; permite após removê-los	borda	Protege quem se candidatou.
Nunca exclui projeto concluído	borda	Preserva o histórico.
Respeita a ordem dos status (não inicia sem selecionar)	borda	A máquina não pula etapa.
Aceita requisição sem corpo; responde ao preflight CORS (204)	resiliência	Bordas de infraestrutura.

9.3 Componente de frontend — o interruptor na parede (RTL, `api.js` mockado)

Cenário	Família	Por que importa
<code>Estrelas</code> : 3 preenchidas com valor 3; <code>onChange(5)</code> no 5º clique; ignora clique em leitura	feliz + borda	O widget central da avaliação.
App : login persiste no localStorage; logout limpa; restaura ao recarregar	feliz	O ciclo de sessão.
App : desloga localmente mesmo se a API de logout falhar	resiliência	Nunca prender o usuário.
App : não quebra se o localStorage estiver corrompido	resiliência	Dado estragado não trava a abertura.
<code>Projetos</code> : não exclui se o usuário cancelar a confirmação	borda	Ação destrutiva exige confirmação.
Páginas mostram a mensagem de erro quando a API falha	borda	Todo caminho de erro também se testa.

9.4 E2E — o morador usando o prédio pronto (Playwright, sem mock)

Spec	O que percorre
logout.spec.js	Logout encerra a sessão de verdade (reload não reloga).
projeto-crud.spec.js	CRUD completo de um projeto pela interface.
fluxo-completo.spec.js	Dois atores: publicar → candidatar → selecionar → andamento → avaliações → concluído.
selecao-candidatos.spec.js	Seleção com remoção de outro concorrente.
freelancer-duas-vagas.spec.js	Uma freelancer conclui duas vagas.
avaliacao.spec.js	A avaliação 360° mútua ao final.

🔗 **A lição para os alunos.** Em todos os níveis testamos o que **deve** acontecer *e* o que **não** deve. É isso que leva a cobertura a 100% de forma honesta — cobrir os dois lados de cada regra.

10 O Makefile: um comando para cada coisa

```
make install    # instala dependências (backend + frontend)
make dev        # sobe backend :3001 + frontend :5173 juntos
make test       # testes unitários/integração dos dois (vitest) — rápido, sem browser
make test-e2e   # testes E2E do Playwright (browser) — sobe os serviços sozinho
make test-all   # tudo: vitest + Playwright
make coverage   # relatórios de cobertura (backend/ e frontend/coverage/index.html)
make stop       # encerra processos nas portas 3001 e 5173-5175
```

✓ **Por que separar test de test-e2e ?** make test é a suíte rápida do dia a dia. make test-e2e abre o navegador e é mais lento. make test-all é o "selo verde" antes de um commit importante.

11 Tags e scripts por tipo de teste

Cada teste recebe uma **tag** no título, para `grep` e para filtrar a execução.

Tag	Tipo de teste	Onde vive
@unitario	regras de negócio	backend/src/regras.test.js
@integracao	API (Supertest)	backend/src/app.test.js
@interface	componente (RTL) + E2E	src/**/*.test.jsx , specs E2E
@e2e	só os E2E de navegador	frontend/e2e/*.spec.js

```
# backend
npm run test:unit      # só regras de negócio
npm run test:integracao # só a API
# frontend
npm run test:interface # só componente/unidade
npm run e2e:e2e        # só E2E de navegador
```

12 Próximo passo: migrar para um banco de dados (Prisma)

Hoje os dados vivem **em memória** (somem ao reiniciar). O passo natural é um banco relacional. A boa notícia: como isolamos tudo em `repositorio.js`, dá para trocar o "armazém" sem tocar na API.

PROMPT 12.A — MIGRAR O REPOSITÓRIO PARA PRISMA (SEM QUEBRAR A API)

@workspace Migre o backend para persistir em banco com Prisma, SEM mudar os endpoints nem os testes de integração. Crie o `schema.prisma` (Usuario, Contrato, Candidatura, Avaliacao com as relações), reescreva `repositorio.js` para usar o Prisma Client mantendo a MESMA interface de funções (`criarUsuario`, `acharContrato`, etc.), e ajuste `server.js` para conectar. Comece com SQLite (dev) e deixe pronto para PostgreSQL. Os testes de `app.test.js` devem continuar passando.

 **Por que dá certo:** os testes de integração testam a API pela borda (HTTP), não o armazenamento. Se eles continuam verdes depois da troca, é a prova de que a migração não quebrou o comportamento.

13 Endpoints da API

Método	Rota	Descrição
POST	/usuarios	Cadastro (endereço/telefone opcionais)
POST	/login	Autentica (retorna o usuário sem a senha)
POST	/logout	Encerra a sessão (stateless)
GET	/usuarios/:id	Usuário + média das avaliações
PATCH	/usuarios/:id	Edita o próprio perfil
POST	/contratos	Publica um projeto aberto
GET	/contratos	Lista projetos (com candidatos e avaliadores)
PATCH	/contratos/:id	Edita título/descrição/contato
DELETE	/contratos/:id	Exclui (só publicado e sem candidatos)
POST	/contratos/:id/candidaturas	Freelancer se candidata
GET	/contratos/:id/candidaturas	Candidatos com reputação
DELETE	/contratos/:id/candidaturas/:fid	Retira candidatura
PATCH	/contratos/:id/selecionar	Seleciona candidato → Em aprovação
PATCH	/contratos/:id/andamento	Confirma acordo → Em andamento
PATCH	/contratos/:id/concluir	Encerra → Concluído
POST	/avaliacoes	Avaliação 360 (em andamento/concluído)

14 Conceitos-chave do projeto

Conceito	O que é	Onde aparece
Contexto para a IA	Regras e stack que o Copilot lê sozinho	.github/copilot-instructions.md
Máquina de estados	Status que só avançam numa ordem válida	app.js (selecionar/andamento/concluir)
Regras de autorização	Quem pode excluir/editar e quando	DELETE guard + botões por papel
Seletores estáveis	data-testid resistente a mudança de visual	botões de ação
Pirâmide de testes	Muitos unitários, alguns integração, poucos E2E	regras / app / specs
Mock só na borda	Dublê para rede/relogio/navegador, nunca a regra	testes de front
Multi-ator no E2E	Dois contextos de navegador no mesmo teste	fluxo-completo.spec.js

15 Referências

- **GitHub Copilot** — docs.github.com/copilot

- **Node.js / Express** — nodejs.org · expressjs.com
 - **Vite / React** — vite.dev · react.dev
 - **Vitest (testes e cobertura)** — vitest.dev
 - **Supertest / Testing Library / Playwright** — github.com/ladjs/supertest · testing-library.com · playwright.dev
-

Projeto Social Garapuvu 2026 · Guia de Prompts do GitHub Copilot